

THOUGHTBOOK

Complete End-to-End Product Workflow & Feature Specification

From first login → dashboard → create post → feed → likes → comments → profiles → notifications → search → settings → logout

Area	Planned functionality
Dashboard	Personalized feed, suggestions, trending/recent content, quick create
Posts	Create, edit, delete, view, like, unlike, save, share
Comments	Add, edit, delete, reply, like
Social	Follow/unfollow users, user profiles, followers/following
Notifications	Likes, comments, follows, replies and other activity
Discovery	Search users/posts and explore topics/tags
Profile	Bio, avatar, user's posts, saved posts, social counts
Settings	Profile, password/security, preferences, account controls

This is a product-level blueprint. Authentication is treated as the entry gate; the main focus is what happens after the user enters ThoughtBook and how every major feature flows through the frontend, API and database.

1. Product Vision

ThoughtBook should feel like a personal social platform for sharing thoughts, ideas, experiences and short written posts. The core loop is simple: **discover** → **read** → **react** → **discuss** → **create** → **return**.

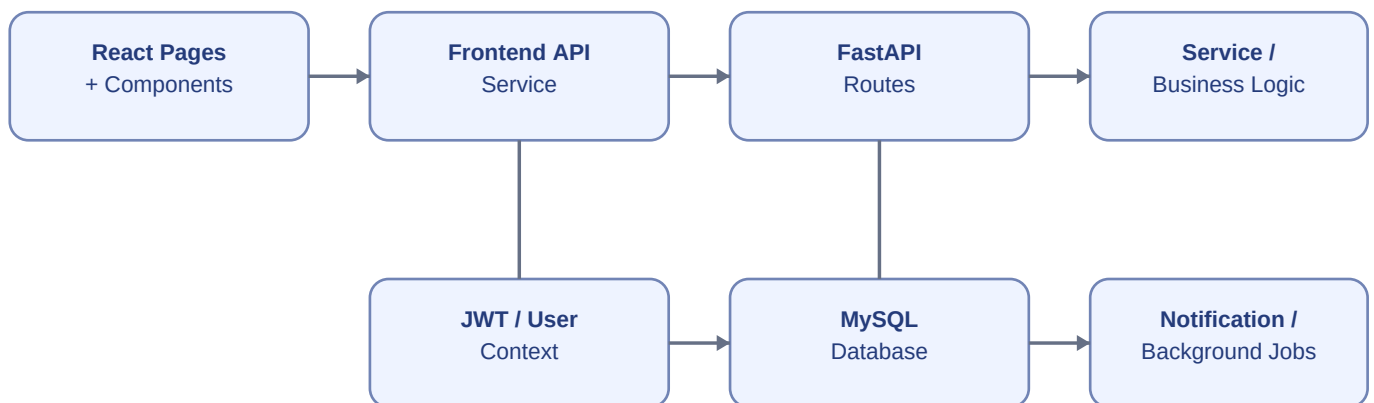
Core user loop

- User opens Dashboard and sees relevant posts.
- User reads a post and can like, comment, save or open the author's profile.
- User creates a thought/post from the Create action.
- Other users interact with the post.
- Post owner receives notifications and can manage the post.
- Users discover more people/content through search, tags and recommendations.

Recommended MVP priority

- **Must have:** Dashboard/feed, create post, view post, edit/delete own post, like/unlike, comments, profile, follow/unfollow, notifications.
- **Should have:** save/bookmark, search, tags, replies, share/copy link, post reporting.
- **Later:** messaging, advanced recommendations, media processing, analytics, moderation dashboard.

2. Complete Application Architecture



Rule: React handles presentation and interaction; FastAPI enforces permissions and business rules; MySQL is the source of truth for persistent application data.

Example request path

Click Like → React → POST /posts/{id}/like → FastAPI verifies user → DB checks like → insert/delete → JSON response → React updates UI

3. Dashboard / Main Navigation

The Dashboard is the user's home after entering ThoughtBook. It should answer: **What is new? What is interesting? What can I do?**

- **Navigation:** Home, Explore, Create (+), Notifications, Saved, Profile, Search and Settings.

- **Desktop:** sidebar navigation + center feed + optional right rail for suggestions/trending topics. **Mobile:** bottom navigation with Home, Explore, Create, Notifications and Profile.
- Keep Create visually prominent because creating thoughts is the primary content-generation action.

4. Dashboard Feed Workflow

- **Feed modes:** For You, Following, Latest/Recent and optional Trending.
- **Post card:** avatar, author, timestamp, title/body preview, tags, like count, comment count, save, share and three-dot menu.
- **Owner menu:** Edit/Delete. **Other-user menu:** Report.
- **UX:** skeleton loading, empty-feed onboarding, retry on failure, pagination/infinite scroll.

5. Create Post — Complete Workflow

- **Composer:** title (optional), body (required), tags, visibility and optional media in a later version.
- User writes → frontend validates → POST /api/posts → FastAPI authenticates/validates → database creates post → API returns post → React inserts it into feed.
- **Example:** POST /api/posts, GET /api/posts/feed, GET /api/posts/{post_id}.
- After publishing, show success feedback and reset/close the composer.

6. Edit / Delete Post Workflow

- **Edit:** Owner opens Edit → composer prefilled → Save → backend verifies ownership → DB updates content and updated_at → UI shows updated post/Edited label.
- **Delete:** Owner chooses Delete → confirmation → backend verifies ownership → DB deletes according to cascade policy → UI removes post.
- **APIs:** PUT /api/posts/{post_id} and DELETE /api/posts/{post_id}.

7. Like / Unlike Workflow

- Click Like → POST /posts/{id}/like → authenticate → check existing like → insert if absent → return liked=true and current count.
- Unlike → DELETE /posts/{id}/like → remove row → return liked=false and updated count.
- Use a unique database constraint on (**user_id**, **post_id**) so one user cannot create duplicate likes.
- Likes can be optimistically updated in React and rolled back if the API fails.

8. Comments & Replies Workflow

- Open post → load comments → type comment → POST /posts/{id}/comments → validate/authenticate → save comment → return comment → update UI without full reload.
- Replies use **parent_comment_id** so comments can form threads.
- Comment author can edit/delete; post owner can optionally moderate comments; users can report inappropriate content.
- **APIs:** GET/POST /api/posts/{post_id}/comments and PUT/DELETE /api/comments/{comment_id}.

9. Save / Bookmark & Share

- Save → POST /posts/{id}/save → bookmark row. Unsave → DELETE /posts/{id}/save.
- Saved page lists the user's bookmarks. Saved content is private unless the product explicitly makes it public.
- Share copies a canonical post URL; optionally use the browser's native share API on mobile.

10. User Profile & Follow Workflow

- **Profile:** avatar, name, username, bio, follower/following counts, Follow button and public posts.
- Follow → backend prevents self-follow and duplicates → creates relationship → updates count → creates notification. Unfollow reverses it.
- **APIs:** GET /api/users/{username}, GET /api/users/{username}/posts, POST/DELETE /api/users/{id}/follow.

11. Notifications Workflow

- Types: post like, post comment, comment reply, follow and optional mention/tag.
- Other user acts → backend completes action → creates notification → recipient sees unread badge → opens notification → navigates to relevant post/profile/comment → mark read.
- **APIs:** GET /api/notifications, PATCH /api/notifications/{id}/read, POST /api/notifications/read-all.

12. Search & Explore

- **Search:** users by username/name, posts by keywords and topics/tags. Debounce search input.
- **Explore:** trending posts, popular/recent tags, suggested users and recent public posts.
- **APIs:** GET /api/search?q=..., GET /api/explore/trending, GET /api/tags/{tag}/posts.

13. Settings & Account Management

- **Edit profile:** name, username, bio, avatar. **Security:** password change for local accounts.
- **Privacy:** visibility options if supported. **Notifications:** preference controls.
- **Logout:** clear frontend authentication state and return to login. **Delete account:** explicit confirmation and defined deletion/anonymization policy.

14. Recommended Database Model

The social workflow needs more than a user table. The following relational model is a strong starting point.

Table	Important fields	Purpose
users	id, email, username, name, bio, avatar_url, created_at	Identity/profile
posts	id, author_id, title, body, visibility, created_at, updated_at	User-created thoughts
likes	id, user_id, post_id, created_at	Post likes; unique user+post
comments	id, post_id, user_id, parent_comment_id, body, created_at, updated_at	Comments/replies
follows	id, follower_id, following_id, created_at	User relationships
bookmarks	id, user_id, post_id, created_at	Saved posts
notifications	id, recipient_id, actor_id, type, post_id, comment_id, is_read, created_at	Activity alerts
tags	id, name	Reusable topics
post_tags	post_id, tag_id	Many-to-many post/topic mapping
reports	id, reporter_id, post_id/comment_id, reason, status, created_at	Moderation/reporting

Key relationships

- One user → many posts, likes, comments and bookmarks.
- One post → many likes and comments.
- Comments can self-reference through parent_comment_id.
- Users have many-to-many follow relationships.
- Posts and tags form a many-to-many relationship.

15. Complete End-to-End User Journeys

Journey A — New user creates their first thought

- Enter ThoughtBook → Dashboard opens.
- Empty feed → onboarding prompt encourages following or creating a thought.
- Click Create → composer opens → enter content → Publish.
- FastAPI validates and stores post → Dashboard displays it.
- User can like, save, share or open comments.

Journey B — Discover and interact

- Feed loads relevant public/following posts.
- Open post → read → like → comment → follow author.
- Post author receives interaction notifications.

Journey C — Manage content

- Owner opens post menu → Edit → save changes.
- Later Delete → confirm → backend checks ownership → removes post.

Journey D — Daily return loop

- Open app → unread notification badge.
- Open notification → jump to related post/profile.
- Return to Home → discover new content → interact or create another thought.

16. Suggested API Map

Feature	Method	Endpoint
Current user	GET	/api/users/me
Feed	GET	/api/posts/feed
Create post	POST	/api/posts
Read post	GET	/api/posts/{id}
Edit post	PUT	/api/posts/{id}
Delete post	DELETE	/api/posts/{id}
Like	POST	/api/posts/{id}/like
Unlike	DELETE	/api/posts/{id}/like
Comments	GET	/api/posts/{id}/comments
Add comment	POST	/api/posts/{id}/comments
Edit comment	PUT	/api/comments/{id}
Delete comment	DELETE	/api/comments/{id}
Save	POST	/api/posts/{id}/save
Unsave	DELETE	/api/posts/{id}/save
Profile	GET	/api/users/{username}
Follow	POST	/api/users/{id}/follow
Unfollow	DELETE	/api/users/{id}/follow
Notifications	GET	/api/notifications
Read notification	PATCH	/api/notifications/{id}/read
Search	GET	/api/search?q=...

17. Suggested Frontend Structure

```
src/ ■■■ pages/ (Login, Dashboard, Explore, CreatePost, PostDetails, Profile, Notifications, Saved, Settings) ■■■ components/ (Navbar, Sidebar, PostCard, CommentSection, CreatePostModal, UserCard, NotificationItem) ■■■ services/api.js ■■■ context/AuthContext.jsx ■■■ App.jsx
```

The exact names can change. The key is separating page UI, reusable components, API communication and authentication/application state.

18. Permission & Business Rules

- Any authenticated user can create a post unless restricted.
- Only the owner can edit/delete a post.
- Only comment owner can edit/delete; post owner may have moderation permission.
- A user can like/save a post at most once.
- A user cannot follow themselves and cannot duplicate a follow.
- Private posts must never be returned to unauthorized users.
- Every protected API action must derive identity from the verified auth context, not a client-supplied user_id.
- Backend authorization is authoritative; hiding a React button is not security.

19. UX States

- Loading: skeletons/spinners.
- Empty: no posts/no notifications/no saved items.
- Error: readable message + Retry.
- Success: lightweight feedback for create/update/delete/save.
- Optimistic likes/bookmarks with rollback on failure.
- Confirmation for destructive actions.
- Responsive desktop/mobile experience.

20. Recommended Build Order

Build in vertical slices so every phase produces a working part of the product.

Phase	Build	Result
1	Dashboard shell + navigation	User can enter and navigate
2	Post CRUD	Create/read/edit/delete thoughts
3	Like/unlike	Basic engagement
4	Comments/replies	Discussion
5	Profile + follow	Social graph
6	Notifications	Feedback loop
7	Save + search + explore	Discovery/retention
8	Settings + privacy	Account control
9	Testing	Reliable system
10	Deployment/security	Production-ready

21. Final Product Flow

Login → Dashboard → Discover posts → Open/read → Like/comment/save/share → Follow users → Receive notifications → Create thought → Publish → Receive engagement → Manage posts → Explore/search → Profile/settings → Logout

The central product relationship is **User → Posts → Likes/Comments/Bookmarks**, while **User ↔ User** handles follows and **Notifications** connects activity back to the affected user.

This is the complete social-content workflow to use as the blueprint for building ThoughtBook beyond the login system.

End of Product Workflow Specification